

Assignment - 1

Kiran Kumar P

1 1-Bit Register vs DFF

1.1 Theoretical Comparison: DFF vs 1-Bit Register

DFF:

At every rising edge of the clock, it copies the value present at its D input to its Q output. This happens automatically every clock cycle, without any additional control. It updates the stored value with whatever input it sees at the clock edge.

1-Bit Register:

A multiplexer is placed in front of the DFF, and the multiplexer is controlled by a load signal. This allows the circuit to decide whether a new value should be written or whether the current value should be kept.

- If $\text{load} = 1$, the multiplexer selects the external input in . The new value is then stored in the DFF at the next clock edge.
- If $\text{load} = 0$, the multiplexer feeds the DFF's current output back into its input. The stored value is written back, preserving it.

Because of this, a register can keep a value unchanged for multiple clock cycles and update it only when required. DFF does not provide this flexibility since it updates its state on every clock edge.

1.2 1-Bit Counter

DFF:

On connecting $D = \text{NOT}(Q)$, the flip-flop will toggle on every clock edge. It runs continuously without any pause or control on when the counting happens.

1-Bit register:

With a 1-Bit register, $\text{in} = \text{NOT}(\text{out})$, and the load signal lets us control the counter:

- $\text{load} = 1$: the counter changes on the next clock edge.
- $\text{load} = 0$: the counter keeps its current value.

By this method, the counting process can be enabled or paused when needed.

2 Byte-Level Access (RAM2)

2.1 Can the architecture use only in, load, and out?

No as we would like to support byte-level operations inside a 16-bit word.

If the chip only has in[16], load, and out[16], then every read or write operation must involve the entire 16-bit word. The single load signal can only indicate whether to write or not, but it cannot specify which part of the word should be updated.

Because of this, the architecture would have no way to perform operations like writing only the upper 8 bits or only the lower 8 bits. Control signals are therefore needed to distinguish between these cases.

2.2 What extra signal is required?

An additional control signal - byte-select signal(1 bit). Let us call it byteSel:

- byteSel = 0: select the lower byte (bits 0-7).
- byteSel = 1: select the upper byte (bits 8-15).

2.3 Advantages and Disadvantages

	8-bit (byte) writes	16-bit (word) writes
Advantages	Allows updating one byte without affecting the other. This is useful for handling characters and strings.	Hardware design is simpler since no byte-selection logic is required. Writing the entire 16-bit value can also be done in a single operation.
Disadvantages	Needs extra control signals (like byteSel) and slightly more complex logic. Writing a full 16-bit value requires two separate byte writes.	Inefficient when only 8 bits need to change, since the whole 16-bit word must be rewritten

Table 1: Comparison of 8-bit vs 16-bit

3 RAM8 Architecture

3.1 Input and Output Signals

Signal	Width	Direction
in[16]	16 bits	Input
load	1 bit	Input
address[3]	3 bits	Input
out[16]	16 bits	Output

Table 2: RAM8 signals

3.2 Address Bits Required

Since RAM8 contains $8 = 2^3$ registers, we need exactly 3 address bits to uniquely index each register. The address values range from 000_2 (register 0) to 111_2 (register 7).

4 Memory Sizing

4.1 Bits in 1 Kilobit (Kb)

$$1 \text{ Kb} = 1024 \text{ bits}$$

4.2 Bits in 1 Kilobyte (KB)

$$1 \text{ KB} = 1024 \text{ bytes} = 1024 \times 8 = 8192 \text{ bits}$$

4.3 Bits in 16 KB

$$16 \text{ KB} = 16 \times 1024 \times 8 = 131,072 \text{ bits}$$

5 Scaling Up Memory

$$1 \text{ KB memory} = 1024 \text{ bytes}$$

5.1 Using RAM8 chips

$$\text{RAM8} = 8 \times 16 = 128 \text{ bits} = 16 \text{ bytes}$$

$$\text{Number of RAM8 chips} = \frac{1024 \text{ bytes}}{16 \text{ bytes/chip}} = 64 \text{ chips}$$

5.2 Using RAM16 chips

$$\text{RAM16 chip} = 16 \times 16 = 256 \text{ bits} = 32 \text{ bytes}$$

$$\text{Number of RAM16 chips} = \frac{1024 \text{ bytes}}{32 \text{ bytes/chip}} = 32 \text{ chips}$$

5.3 Using RAM64 chips

RAM64 chip stores $64 \times 16 = 1024$ bits = 128 bytes

$$\text{Number of RAM64 chips} = \frac{1024 \text{ bytes}}{128 \text{ bytes/chip}} = 8 \text{ chips}$$

6 Address Line Management

6.1 RAM8 vs RAM64

For a 1 KB memory (512 sixteen-bit words), we need $\log_2(512) = 9$ address bits in total.

Using RAM8 chips (64 chips):

- **Upper 6 bits** (address[3..8]): select which of the 64 RAM8 chips is active (chip select via DMux).
- **Lower 3 bits** (address[0..2]): select which register within the chosen RAM8 chip.

Using RAM64 chips (8 chips):

- **Upper 3 bits** (address[6..8]): select which of the 8 RAM64 chips is active.
- **Lower 6 bits** (address[0..5]): select which register within the chosen RAM64 chip.

The splitting changes, but the total number of address bits stays the same.

7 Extra Credit: Architectural Impact

7.1 Small Blocks (RAM8) vs Large Blocks (RAM4K)

7.1.1 Hardware Complexity and Propagation Delay

Using many small blocks (RAM8):

Constructing large memory from RAM8 units requires many layers of decoders and multiplexers. For example, building RAM16K would require $16,384/8 = 2,048$ RAM8 chips arranged hierarchically. Each additional layer adds extra DMux/Mux logic, which increases circuit complexity and propagation delay.

Using larger blocks (RAM4K):

If we use RAM4K blocks instead, the hierarchy becomes much simpler. RAM16K can be built using only four RAM4K chips with a single 4-way selection stage. This reduces the number of logic stages and results in faster memory access.

7.1.2 Relevance to AI/ML Workloads

AI/ML workloads operate on large matrices and tensors, requiring frequent access to contiguous memory regions. Larger memory blocks reduce address decoding overhead and improve access latency, which is important when performing millions of memory operations during training or inference.

Smaller blocks are useful for highly parallel structures such as register files, but for bulk data movement in neural network computations, larger memory blocks are generally more efficient.

7.2 Manufacturing and Cost: 1024 RAM8 vs 8 RAM1K

1024 RAM8 chips would be more expensive to design and manufacture for 16 KB of memory than 8 RAM1K chips.

The reasons are:

1. **Interconnect overhead:** 1024 chips require far more wiring for data, address, and control signals compared to just 8 chips.
2. **Address decoding:** Selecting 1024 chips needs a 10-bit decoder, while 8 chips only require a simple 3-bit decoder.
3. **Power usage:** Each chip has its own I/O drivers and leakage power, so 1024 chips consume much more power than 8.
4. **Area efficiency:** Larger memory blocks use silicon more efficiently since control and I/O overhead is shared across more storage cells.

8 Wallace Tree Multiplier Design

8.1 Partial Product Generation

Let the two 16-bit inputs be $A = a_{15}a_{14} \dots a_0$ and $B = b_{15}b_{14} \dots b_0$.

Each bit of A is multiplied with each bit of B using AND gates to form the partial products. Thus, we generate

$$p_{i,j} = a_i \cdot b_j, \quad 0 \leq i, j \leq 15$$

8.2 Wallace Tree Reduction

Reduction stages:

1. **Stage 1** (5 CSAs): The 16 rows are divided into five groups of three rows each, leaving one remaining row. Each CSA compresses its three inputs into a sum and carry vector. After this stage, we obtain the outputs of the five CSAs along with the leftover row.
2. **Stage 2** (3 CSAs): The vectors produced in the first stage are again grouped into triplets and reduced using three CSAs.
3. **Stage 3** (2 CSAs): The remaining vectors are further reduced using two CSAs.
4. **Stage 4** (1 CSA): Another compression stage reduces the vectors further.
5. **Stage 5** (1 CSA): The final CSA reduces the remaining vectors to two outputs.

Only two 32-bit vectors remain: a sum vector and a carry vector.

8.3 Final Carry-Lookahead Addition

The final step is to add the two remaining vectors to produce the 32-bit product. This is done using a 32-bit Carry-Lookahead Adder (CLA). The CLA computes carries in parallel.

Depth comparison:

- **Ripple approach:** If the 16 rows were added sequentially, we would need 15 full 32-bit additions. Since ripple-carry adders propagate carries linearly across the bits, each addition introduces a delay proportional to the word size. This leads to a total delay on the order of $O(15 \times 32)$.
- **Wallace tree approach:** The CSA stages operate in parallel and have constant delay. With only a few reduction stages followed by a fast CLA, the total delay becomes approximately $O(\log n)$ for a 32-bit result.

8.4 Signed Multiplication

1. **Compute absolute values:** If the sign bit is 1, negate the input ($\neg x + 1$). This uses Not16, Add16, and Mux16.
2. **Unsigned multiply:** Feed $|A|$ and $|B|$ into the Wallace tree multiplier.
3. **Determine result sign:** $\text{sign} = a_{15} \oplus b_{15}$.
4. **Conditionally negate:** If $\text{sign} = 1$, negate the 32-bit product using XOR gates and a CLA to add 1.

9 Adder Choice Justification

Adder Type	Delay (32-bit)	Hardware Cost
Ripple Carry	$O(32)$	$O(32)$ gates
Carry Lookahead	$O(\log 32) = O(5)$	$O(32 \log 32)$ gates

Table 3: Comparison of adder delay and cost for 32-bit operands

The CLA trades additional hardware for roughly $6\times$ lower delay, which is worthwhile since the final addition is on the critical path.

10 Instruction Encoding for Multiplication

10.1 Hack ALU Control Bits

The Hack ALU uses six control bits: **zx**, **nx**, **zy**, **ny**, **f**, **no**. These produce $2^6 = 64$ possible combinations, of which 18 are documented as standard operations.

10.2 Chosen Encoding

We assign multiplication to the control-bit pattern:

zx	nx	zy	ny	f	no	Operation
1	1	1	1	0	0	$x \times y$ (multiplication)

Binary: 111100.

10.3 Why This Does Not Conflict?

In the standard ALU, the encoding 111100 would compute:

- $zx = 1, nx = 1$: $x \rightarrow 0 \rightarrow \text{NOT}(0) = 0xFFFF$
- $zy = 1, ny = 1$: $y \rightarrow 0 \rightarrow \text{NOT}(0) = 0xFFFF$
- $f = 0$: $\text{AND} \rightarrow 0xFFFF \& 0xFFFF = 0xFFFF$
- $no = 0$: $\text{output} = 0xFFFF = -1$

This is **not** one of the 18 documented Hack ALU operations. The documented way to produce -1 is 111010 (different ny , f bits). No standard program uses 111100, so repurposing it for multiplication is safe and backward-compatible.

10.4 ALU Implementation

The modified ALU detects the multiplication pattern using:

$$\text{mulSel} = zx \cdot nx \cdot zy \cdot ny \cdot \bar{f} \cdot \bar{no}$$

When $\text{mulSel} = 1$, a multiplexer selects the multiplier's output (lower 16 bits of $x \times y$) instead of the standard ALU result. The multiplier receives the *original* x and y inputs, bypassing the zero/negate preprocessing.

11 Special Case Optimization

11.1 Multiplication by Powers of Two

When one multiplicand is 2^N , the multiplication $x \times 2^N$ is equivalent to a left shift:

$$x \times 2^N = x \ll N$$

In hardware, a left shift requires no arithmetic at all - it is simply a rewiring of bit positions.

11.2 Multiplication by $2^N + 1$

When the multiplicand is $2^N + 1$:

$$x \times (2^N + 1) = (x \ll N) + x$$

This requires only one shift and one addition - dramatically simpler than a full multiplier, which needs 256 AND gates and 11 CSA stages.

Favorite Song: Chandrachooda